

repo_name	role_name	files_included	evidence
rustdesk/rustdesk	Session	src/client.rs	Authentication, session state, connection lifecycle, OS input injection, and UI/network coordination. Arc<RwLock<LoginConfigHandler>> acts as the core shared session object and is accessed across UI threads, async network tasks, IPC handlers, and OS worker threads. Shared coordination is used to maintain consistent session/auth/input state across long-lived client execution contexts.
	coordination layer		
rustdesk/rustdesk	Connection	src/rendezvous_me	Coordinates peer registration, NAT traversal, relay fallback, PK/key synchronisation, and distributed connection lifecycle management across async tasks and network handlers. Shared state is used for correctness and coordination decisions, including deduplication, latency state, identity consistency, relay routing, and restart/shutdown control across distributed network execution contexts.
	coordination layer	diator.rs	
rustdesk/rustdesk	Runtime	src/server.rs	Shared server runtime coordinating active remote desktop sessions, service lifecycle management, background workers, IPC services, and connection control flow. Arc<RwLock<Server>> -> runtime registry shared across Tokio tasks, OS threads, relay handlers, and service threads. Atomic flags coordinate shutdown, restart, and registration lifecycle state.
	coordination layer		
rustdesk/rustdesk	I/O coordination	src/client/io_loop.rs	Coordinates shared networking, video streaming, clipboard synchronisation, transfer jobs, and media/control flow across async tasks, decoder/render workers, UI handlers, and platform callbacks. Shared-memory state is used to coordinate runtime observability, telemetry, frame queues, shutdown flags, permissions, and streaming control between concurrent I/O and rendering related execution contexts.
	layer		
dani-garcia/vaultwz	Shared	src/config.rs	Shares configuration state: Config: LazyLock<Config> provides global singleton access. inner: RwLock<Inner> supports high-read rare-write runtime configuration. SKIP_CONFIG_VALIDATION: AtomicBool is a lock-free runtime behaviour switch. DashMap cache and LazyLock singletons share IO backends, templates, and version metadata across the async runtime. Configuration updates replace snapshots instead of mutating shared state in place.
	configuration		
dani-garcia/vaultwz	system		
	Resource	src/http_client.rs	Primary responsibility is managing external resources (HTTP client, DNS resolver, regex cache), not controlling system logic or state flow: LazyLock shared request client and DNS resolver avoid rebuilding expensive networking infrastructure. INSTANCE: LazyLock<Arc<CustomDnsResolver>> shares resolver state across tasks. COMPILED_REGEX: Mutex<Option<(String, Regex)>> caches compiled regexes and updates when configuration changes. Shared immutable clients and connection pools minimise runtime cost and reuse DNS/TLS state.
dani-garcia/vaultwz	coordination layer		
	Runtime	src/main.rs	manage(pool), manage(Arc::clone(&WS_USERS)), runtime.spawn(... pool.clone()), and CONFIG.set_rocket_shutdown_handle(instance.shutdown()) -> shared application runtime coordination. Shared DB pools, websocket registries, shutdown handles, scheduler jobs, and async runtime resources are cloned into worker tasks and signal handlers through Arc ownership.
dani-garcia/vaultwz	coordination layer		
	Caching layer	src/api/icons.rs	static CLIENT: LazyLock<Client> reuses pooled HTTP/TLS state across favicon requests. Arc<Jar> shares cookie storage between async tasks. Jar(std::sync::RwLock<CookieStore>) allows concurrent reads with some writes for cookie updates. ICON_SIZE_REGEX and PRIORITY_MAP act as immutable shared caches. Shared networking state and parsing caches reduce recomputation and allocation during concurrent downloads
dani-garcia/vaultwz	Database	src/db/mod.rs	Arc<Semaphore> limits concurrent database users across Tokio tasks. conn: Arc<Mutex<Option<PooledConnection<DbConnManager>>>> serialises mutable Diesel connection access while allowing ownership transfer into blocking workers. spawn_blocking ... separates blocking DB work from async runtime threads. ACTIVE_DB_TYPE: OnceLock<DbConnType> initialises backend metadata once and shares it globally.
	coordination layer		

	Runtime	native/src/core/dae	Global daemon singleton coordinating runtime state across worker threads(MAGISKD functions as global runtime container:), pub static MAGISKD: OnceLock<MagiskD> = OnceLock::new() shares daemon state across the whole process, shared SQLite connection, manager metadata, boot stage state, zygisk runtime state, cached SU info, and module metadata coordinated through Mutex, AtomicBool, AtomicArc and OnceLock. all worker threads share one daemon object and synchronise access to mutable subsystems. coordinates shared runtime subsystem state and lifecycle across the whole process, not just individual shared variables -> more like runtime coordination layer
topjohnwu/Magisk	coordination layer	mon.rs	
	Logging system	native/src/core/loggi	Shared logging infrastructure coordinating concurrent log delivery between producer threads and dedicated logging daemon thread, static MAGISK_LOGD_FD: Mutex<Option<Arc<File>>> serialises shared logfile ownership while writes occur lock-free after Arc clone. static ZYGISK_LOGD: AtomicI32 safely publishes/reset shared FD state, concurrent writers rely on PIPE_BUF kernel atomicity to avoid userspace locking
topjohnwu/Magisk		ng.rs	
		native/src/core/thre	Shared daemon thread pool coordinating worker lifecycle and task dispatch. static THREAD_POOL: LazyLock<ThreadPool> initialises one shared pool across threads, ThreadPool contains Condvar task_is_some/task_is_none and Mutex<PoolInfo> shared scheduler metadata, producers and worker threads coordinate sleep/wakeup and task handoff through monitor-style mutex + condition variable synchronisation
topjohnwu/Magisk	Resource coordinati	ad.rs	
		native/src/core/su/d	Shared cached SuInfo object reused across concurrent SUPERUSER requests, pub cached_su_info: AtomicArc<SuInfo> provides lock-free shared cache visibility across daemon worker threads, access: Mutex<AccessInfo> protects mutable authorisation state while the rest of SuInfo remains shared and read-heavy, cached policy objects reused to avoid repeated DB-derived policy reconstruction
topjohnwu/Magisk	Caching layer	aemon.rs	
		alacrity/src/logging.	Shared logfile writer, stdout writer, event-loop proxy, logfile-created flag, cached extra log targets, multiple threads can log concurrently so writes must be serialised, Mutex<OnDemandLogFile>, Mutex<LineWriter<Stdout>>, Mutex<EventLoopProxy<Event>> serialise concurrent logging and UI notifications, Arc<AtomicBool> tracks logfile creation without locking, EXTRA_LOG_TARGETS: OnceLock initialises immutable shared config once
alacrity/alacrity	Logging system	rs	
		alacrity/src/rende	OpenGL loader initialisation and immutable extension cache shared across renderer threads, GL_FUNS_LOADED: AtomicBool ensures OpenGL functions are initialised only once, OPENGL_EXTENSIONS: OnceLock<HashSet<...>> provides immutable shared extension state, rendering remains mostly thread-confined through &mut self so shared memory is minimised
alacrity/alacrity	Rendering coordination layer	r/mod.rs	
		alacrity_terminal/sr	Main PTY event loop coordinating terminal I/O and rendering, shared terminal state accessed by PTY thread and renderer through Arc<FairMutex<Term<U>>>, PTY thread reads shell output while renderer safely reads terminal state, fairness controls prevent renderer starvation under heavy PTY throughput, channels used for commands and event wakeup signalling. Contains event loop, but the architectural pattern is more geared towards I/O coordination
alacrity/alacrity	I/O coordination laye	c/event_loop.rs	
		alacrity_terminal/sr	FairMutex<T> controls shared-memory access fairness and prevents starvation, data: Mutex<T> protects shared object while next: Mutex<()> coordinates lock acquisition order, threads first acquire next then data to serialise contention, supports fair and unfair locking modes for throughput vs fairness tradeoffs
alacrity/alacrity	Synchronisation prim	c/sync.rs	
		alacrity_terminal/sr	Cross-thread Windows process monitoring and event notification, Child process handle and poller registration shared across callback and watcher threads, AtomicPtr<c_void> used because WinAPI callback cannot block or lock, Arc<Mutex<Option<Interest>>> protects mutable poller registration state while exit events are delivered back into the application event loop
alacrity/alacrity	OS integration bridge	c/rs	
			pending_requests: Mutex<HashMap<seq, Sender>>, match server responses to originating client request, many
helix-editor/helix	Request coordinatio	helix-dap/src/transpo	async tasks read/write request map, insert/remove pending requests concurrently across send/recv tasks

helix-editor/helix	Rendering coordinat	helix-event/src/redra	RENDER_LOCK: RwLock<()>; prevents rendering during async work. Renderer acquires write lock in start_frame, background tasks block frame rendering. redraw requests are event-driven
			global event bus per runtime, registering + dispatching hooks. Registry contains events and handlers maps.
helix-editor/helix	Event dispatch syste	helix-event/src/regist	central coordination of event registration and execution. read-heavy dispatch, write-light registration
helix-editor/helix	Runtime coordinatio	helix-event/src/runtir	hashmap mapping runtime IDs to runtime specific singleton instances, prevents integration tests sharing globals, runtime specific state partitioning, read lock for lookup, write lock on miss
			main application loop that coordinates input, rendering, jobs. Config shared via ArcSwap, single-owner event
helix-editor/helix	Execution loop	helix-term/src/applici	loop, async tasks spawned for UI, jobs, LSP. Message passing replaces shared mutation
			transactional document state, incremental updates, rope snapshots, savepoints protected by
helix-editor/helix	State synchronisatio	helix-view/src/docum	Mutex<Transaction>, ArcSwap config and syntax loader, weak references for lifecycle management
farion1231/cc-swite	Runtime coordinatio	src-tauri/src/lib.rs	runtime file, application bootstrapper, initialise plugins, start background workers, registers events, sets up u. Centralised async shared state. Services accessible through AppState, cloned via Arc, coordinated through async primitives and database serialisation.
		src-tauri/src/commands	Multiple async tasks must view and update one provider health view so routing logic can reject unhealthy providers. Shared-memory async coordination: lock-free atomic coordination for counters and permit management. Many tasks can share the same breaker, and all mutations need to be thread-safe. -> Synchronise
farion1231/cc-swite	State synchronisatio	/webdav_auto_sync.rs	shared provider-health state, coordinate concurrent access to counters/state transitions, ensuring visibility of Open/HalfOpen/Closed state across tasks
farion1231/cc-swite	I/O coordination laye	src-tauri/src/proxy/circu	HTTP proxy server handling API requests, routing, failover, circuit breaker. Maintain consistent routing across requests, allow runtime config updates, expose shared server status. Tokio async runtime, task per connection, worker threads
		src-tauri/src/proxy/resp	response processing. Handling of streaming/non-streaming responses, Sse usage, timeout coordination, async logging, safe request state propagation. SSE collecto: incremental parse -> shared accumulation -> atomic finalize -> detached persistence. Logging pipeline is decoupled, improving responsiveness because DB writes
farion1231/cc-swite	Request coordinatio	onse_processor.rs	never block response streaming.
farion1231/cc-swite	Resource coordinatio	src-tauri/src/proxy/serv	Prevent concurrent sync corruption, coordinate database imports/exports, avoid overlapping manual/automatic sync. shared-state coordination using async mutual exclusion. Concurrency allowed at application level but one mutex used for synchronisation
		src-tauri/src/services/w	Multiple components must trigger sync while avoiding duplicate uploads and recursive sync loops. Concurrent producers send change signals, async worker consumes and batches events. Actor/worker-style architecture,
farion1231/cc-swite	Event dispatch syste	ebdav_auto_sync.rs	channels, bounded channel acts as flag queue, debounce merges bursts of events
sharkdp/fd	Runtime coordinatio	src/main.rs	Main, parse cli, build config, compile regexes. Prepares shared state, packages data for worker threads. main thread: builds config, regex objects, filters, calls walk::scan(). command: command.map(Arc::new) wraps the command in Arc before storing it in Config -> this file bootstraps runtime state and prepares shared resources for concurrent execution.
		src/walk.rs	parallel filesystem scanning, filter entries and send matching results between threads. producer/consumer coordination between worker/walker threads and receiver threads. WalkParallel spawns filesystem walker workers. communication between threads uses channels crossbeam_channel::bounded. Uses batching to
sharkdp/fd	I/O coordination laye		reduce channel traffic -> coordinates filesystem traversal and result flow between worker threads.

y

y

		crates/nu-protocol/src/pipeline.rs	Threads register handlers. Event handlers. Multiple threads need to register, remove, execute handlers through a shared registry. Shared reads during dispatch and synchronised writes during registration/removal via mutex locking. Arc<Mutex<Vec<(usize, Handler)>>> -> shared ownership across threads + synchronised mutable access. -> acts as a central event/handler registration and dispatch system
nushell/nushell	Event dispatch system	handlers.rs	
		crates/nu-utils/src/sync/keyed_lazy_lock.rs	for caching, initialise once, thread safe. Avoid repeating expensive computation + safely sharing initialised values across threads. Given a key, you can compute a value once, store it, and let all threads reuse that same initialised value. RwLock protects cache structure while OnceLock serialises initialisation per key -> implements a reusable lazy cache
nushell/nushell	Caching layer		
		crates/nu-plugin-core/src/interface/stream/mod.rs	streamwriter produces stream data, streamreader consumes it, streammanager tracks active streams. Multiple threads must coordinate stream ownership, acknowledgements, cancellation, and backpressure. Combines message-driven communication with shared coordination, uses backpressure windowing similar to TCP. writer writes data -> transport messages -> streammanager handles message -> reader receives chunk + consumes + sends ack -> coordinates request/response-style streaming interactions with acknowledgements and lifecycle management
nushell/nushell	Request coordination		
		crates/nu-protocol/src/engine/engine_state.rs	runtime state. Shared across runtime components and background jobs while allowing safe concurrent access. Layered concurrency design: immutable shared runtime metadata stored in Arc, mutable coordination state protected with Mutex. jobs tracks background jobs/active tasks/thread jobs. mailbox: Arc<Mutex<Mailbox>>, root_job_sender: Sender<Mail> -> central runtime state container coordinating jobs, REPL state, caches, signalling, and runtime services.
nushell/nushell	Runtime coordination		
		crates/nu-command/src/network/http/client.rs	coordinate HTTP client pool + async request cancellation/state sharing. Global pooled ureq::Agent. Avoid rebuilding HTTP clients/connectors repeatedly. RwLock<Option<Arc<Agent>>> -> many threads can own the same client safely. the agent is the reusable http connection pool -> coordinates shared access to pooled HTTP client/network resources across concurrent requests.
nushell/nushell	Resource coordination		
		lapce-app/src/editor_tabs.rs	Uses reactive programming and shared ownership rather than explicit thread synchronisation, combines derived state with shared mutable UI signals, RwSignal is reactive mutable state, so shared variable + automatic UI updates -> this file primarily coordinates reactive editor/UI rendering state
lapce/lapce	Rendering coordination		
		lapce-app/src/plugin.rs	UI thread reads/writes, worker threads return results via callbacks, Reactive shared-state coordination with message style callbacks -> it synchronises plugin installation and UI-visible plugin state across threads.
lapce/lapce	State synchronisation		
		lapce-proxy/src/dispatch.rs	notifications are sent out when file changed, terminal resized, plugin updated, git commit triggered. Combines message passing with shared mutable coordination state -> it dispatches and coordinates filesystem/editor events between subsystems
lapce/lapce	Event dispatch system		
		lapce-proxy/src/plugin/lsp.rs	lsp client managing external language server process, write and read server, IO and RPC flow rely mainly on channels -> coordinates lifecycle and communication with external LSP server connections
lapce/lapce	Connection coordination		
		lapce-rpc/src/core.rs	Allows multiple threads to generate unique request IDs and match async responses back to waiting callers, pending shared: table of pending requests waiting for responses -> coordinate request/response matching
lapce/lapce	Request coordination		
		yazi-core/src/tasks/tasks.rs	background task that updates progress. Uses snapshotting pattern, minimises lock duration -> coordinates task scheduler state and runtime task monitoring
sxyazi/yazi	Runtime coordination		
		yazi-dds/src/server.rs	TCP server with multiple clients with message routing, broadcast updates, and async I/O handling. shared registry access.-> manages connected peers and routes messages between them
sxyazi/yazi	Connection coordination		
		yazi-dds/src/pubsub.rs	pubsub event routing for lua plugin, dispatches events to multiple targets. Registry as routing table, snapshot, no long held locks -> coordinates event subscription and dispatch routing
sxyazi/yazi	Event dispatch system		

y

y

sxyazi/yazi	Caching layer	yazi-runner/src/loader/loader.rs	plugin loader, cache for lua module. Read-heavy cache lookups (cache.read()), rare write -> mainly provides shared cached plugin/module reuse
sxyazi/yazi	Runtime coordinator	yazi-scheduler/src/scheduler.rs	scheduler: creates tasks, assigns ids, routes them into worker queues. shared memory used for coordination -> The file coordinates scheduling and lifecycle management of worker tasks
sxyazi/yazi	Resource coordinator	yazi-scheduler/src/workers.rs	Many concurrent async workers can coordinate task lifecycle while reusing subsystem services across threads. Separation between execution and coordination -> coordinates shared worker subsystems, cancellation, and execution resources
firecracker-microvm	Request coordinator	src/firecracker/src/api/vmm.rs	Match API requests to VMM actions and returning responses. Channels, EventFd, and controller logic exist to coordinate request/response flow between API and VMM threads
firecracker-microvm	Runtime coordinator	src/firecracker/src/microvm.rs	Bootstraps Firecracker, creates subsystems, starts threads, API server, VMM, and event manager
firecracker-microvm	Runtime coordinator	src/vmm/src/builder.rs	Assembles VM, devices, memory, VMM, thread infrastructure. Coordinates subsystem construction
firecracker-microvm	Runtime coordinator	src/vmm/src/lib.rs	Owns Vmm object, vCPU lifecycle, device manager, snapshots, event handling, VM control operations
firecracker-microvm	OS integration bridge	src/vmm/src/vstate/vcpu.rs	Bridges runtime logic to KVM vCPU threads. Translates runtime commands into KVM execution and synchronises vCPU execution
firecracker-microvm	Resource coordinator	src/vmm/src/vstate/vmm.rs	Manages shared VM resources. Guest memory, MMIO bus, interrupt routing, resource allocation, KVM memory slots. Coordinate access to shared VM resources
casey/just	Shared configuration	src/analyzer.rs	Builds Justfile configuration model used by the rest of the program and validates it. There is only Arc usage and its main role is constructing shared configuration/state before execution
casey/just	Shared configuration	src/dependency.rs	It is a data structure representing resolved dependencies inside the execution model. Coordination is part of the shared configuration/runtime model. Build and maintain shared execution configuration
casey/just	Runtime coordinator	src/justfile.rs	Evaluates scopes, resolves execution context, runs recipes, runs dependencies, spawns parallel workers, manages execution order, and coordinates interaction between multiple subsystems.
casey/just	State synchronisation	src/ran.rs	Maintain consistent shared execution state (has the recipe already run) across threads
casey/just	OS integration bridge	src/signal_handler.rs	Receives OS callbacks, tracks child processes, forwards signals, and translates OS events into runtime actions

y